

FinLearn Guard — GUIDE BOOK (for the project show)

Read this top to bottom once tonight. In class, jump straight to Section 5 (live-edit recipes) when she asks you to change something.

1. What this project is (say this in 30 seconds)

"FinLearn Guard is one website that detects cyber threats AND teaches the fix instantly. A threat comes in, the system detects it with rules plus free AI, alerts everyone live, and the Support page explains the fix by text, voice, and video. Website, detection, and support are one server, one database — that closed loop is the main idea."

Keywords for marks: FastAPI, Jinja2 templates, REST API, WebSocket realtime, MongoDB Atlas (SQLite fallback), OpenRouter AI, edge-tts voice, moviepy video, pytest 3/3, Render deploy.

2. How to run it (do this before class)

1. Double-click `../live/start_live.bat` (a black window opens — KEEP it open).
2. Open in browser:
3. Dashboard: `http://127.0.0.1:8000/`
4. Support: `http://127.0.0.1:8000/support`
5. If the page looks old, press Ctrl+F5 once (fresh load).

3-minute demo path: Dashboard hero → click "Simulate Attack" → feed row pops without refresh → CLICK the row → "How it was caught" shows 5 steps (use "Next step" to walk 1-by-1) → go to /support, ask the OTP question → Steps tab, Voice tab (play), Video tab (play) → say "pytest 3/3 proves it" (run `.venv\Scripts\python -m pytest tests -q`).

3. Folder map — what each file does (one line each)

<code>../live/start_live.bat</code>	double-click to start the server before class
<code>src\main.py</code>	the server itself (starts everything)
<code>src\web\routes_web.py</code>	the 4 PAGES: / /support /users /devices
<code>src\api\routes_api.py</code>	the DATA doors: POST /api/threats, /api/support/ask, /api/emplo
<code>src\api\ws.py</code>	the LIVE wire (/ws/alerts) – feed pops without refresh
<code>src\detection\rules.py</code>	danger-word list + weights → score 0..1 (OFFLINE, always works)
<code>src\agents\detector.py</code>	rules first, then AI verdict (needs internet + key)

```

src\agents\alerter.py    saves the event + broadcasts it live (the 5 pipeline steps)
src\agents\support.py    support guide writer (problem + steps + voice script)
src\llm\client.py        talks to the free AI (OpenRouter); error if key missing
src\llm\prompts\*.txt    instructions given to the AI (edit wording here, NOT in code)
src\media\voice.py        makes the mp3 (edge-tts, free)
src\media\video.py        makes the mp4 (moviepy; slide cards if ffmpeg missing)
src\storage\database.py  SQLite file + Atlas cloud switch, 3 demo employees seeded
templates\layouts\base.html  topbar + background, shared by EVERY page
templates\pages\dashboard.html  hero + live feed + inspector + graphs + employees
templates\pages\support.html  ask box + Steps/Voice/Video answer
templates\pages\users.html    all-users table page
templates\pages\devices.html  device agent status page
static\css\style.css        ALL looks: colors ($0 tokens), buttons ($3), inspector ($11)
static\js\app.js            ALL dashboard behavior: feed, inspector, graphs, auto-feed
tests\                      3 tests: smoke + detection + support (pytest proves it works)
.env                        secrets (AI key + database URL) – NEVER show on screen
../instructor/              DEMO_SCRIPT.txt + ARCHITECTURE.txt + API_TABLE.txt + REPORT_

```

4. GOLDEN RULE — refresh vs restart (memorize this)

- HTML / CSS / JS (`templates\` , `static\`) → just SAVE the file and REFRESH the browser (Ctrl+F5). Server keeps running. No restart.
- Python (`src\*.py`) → SAVE, then RESTART the server: close the black window (or Ctrl+C inside it), **double-click** `start_live.bat` again, wait 5 seconds, refresh. Python code loads only at startup. ▶
- `.env` secrets → SAVE + RESTART (same as Python).
- If she asks "change X" and X is TEXT, COLOR, or BUTTON → it is always HTML/CSS → save + refresh, done in 10 seconds. Smile.

5. LIVE-EDIT PLAYBOOK — when she says "change it now"

Open the file in VS Code, find the text, type, save, refresh browser. Each recipe below gives: FILE → FIND → TYPE → SEE.

E1. Change the big headline

FILE: `templates\pages\dashboard.html` → FIND: Threat in. Lesson out. → TYPE your words → SEE: refresh / . (Same file: `.sub` line = the grey subtitle under it.)

E2. Change any button text

FILE: same `dashboard.html` → FIND: `Simulate Attack` (or `Support`, `Test this`, `Next step`) → TYPE new words, keep the quotes/tags around it → SEE: refresh. Support page buttons live in `templates\pages\support.html` (`Diagnose & guide me`, `Steps`, `Voice`, `Video`).

E3. Change colors / theme

FILE: `static\css\style.css` line 26 (`:root{...}`) → the tokens: `--cyan` (main bright), `--vio` (gradient end), `--rose` (red/danger), `--amber` (medium), `--lime` (safe/green), `--bg` (page background). → TYPE a new hex, e.g. `--cyan:#00ff88` → SEE: refresh. Whole site recolors at once — big applause, 10 seconds.

E4. Change the topbar (site name / links)

FILE: `templates\layouts\base.html` → FIND: `FinLearn Guard` (brand) or the `<nav>` links → TYPE → SEE: refresh. Affects EVERY page at once.

E5. Add a new block on the dashboard

FILE: `templates\pages\dashboard.html` → copy any whole `<section class="card">...</section>` block, paste below it, change its `<h2>` title and inner text → SEE: refresh. New panel appears. (Styling comes free from `.card`.)

E6. Change the Support page question hint or tabs

FILE: `templates\pages\support.html` → FIND: `placeholder="e.g. I got a mail..."` (hint text) or the tab buttons → TYPE → SEE: refresh.

E7. Add YOUR OWN demo attack sample

FILE: `static\js\app.js` → FIND: `const SAMPLES = [` → add one line inside, e.g. `{ title: 'Free gift claim now', body: 'Click http://gift.example with your OTP.' },` → SEE: refresh, click "Simulate Attack" until yours fires. No restart (it is JS).

E8. Change danger words or weights (detection logic)

FILE: `src\detection\rules.py` → each line = `(word-pattern, weight)`. Raise a weight (e.g. `0.15` → `0.30`) or add a line like `(r"bonus|reward", 0.20),` → SAVE + RESTART server (Section 4) → test with "Test this". If she asks "how does detection work", point at this file: words match → points add up → score 0..1 → AI confirms. Safety rule in `src\agents\detector.py` line 50: `score ≥ 0.6` can never be called "safe" even if the AI says so.

E9. Employees (she asks "add a person / change names")

EASY WAY (live, no code): use the "+ Add person" form on the dashboard — it saves instantly. CODE WAY: `src\storage\database.py` lines 64-66 (seed names) → but seeds apply ONLY to an empty database, so prefer the form in class.

E10. Change what the AI is told (prompts, no code)

FILE: `src\llm\prompts\detector.txt` or `support.txt` → plain English instructions → SAVE + RESTART → next answer follows the new wording. Good answer if she asks "where is the AI logic": "the prompt files".

E11. Add a whole new page (only if she pushes)

1. `templates\pages\` → copy `users.html` to `newpage.html`, change text.
2. `src\web\routes_web.py` → copy a 3-line `@router.get(...)` block, change path + filename.
3. `templates\layouts\base.html` → add `<a href="/newpage">` in `<nav>` .
4. SAVE + RESTART → open `/newpage` . Four steps, say them out loud.

6. If something breaks in class (backup lines)

- Port busy / bat closes instantly → another server holds :8000. In PowerShell: find the listener PID on port 8000 and Stop-Process it, then double-click the bat again.
- Support says "error" / verdict=error → AI key missing or no net. Say: "verdict=error by design — it refuses to guess without the key." Rules score + signals still show. Set OPENROUTER_API_KEY in `.env` + restart when net is back.
- Video shows "needs moviepy" → audio + steps still work. Say: "step cards below work without ffmpeg." (moviepy IS installed here, so this should not happen on this PC.)
- Feed looks frozen → check the topbar pill: green LIVE = connected, "reconnecting..." = server window closed → reopen the bat.
- Page shows OLD code after your edit → Ctrl+F5 (hard refresh). Server sends no-store, but browsers cache JS/CSS filenames — that is why we bump `?v=` in `base.html` after static edits.

7. Questions she may ask (one-line answers)

- "How is this realtime?" → POST saves → WebSocket `/ws/alerts` pushes to all open browsers instantly, no refresh (`src\api\ws.py`).
- "Where does the AI run?" → OpenRouter cloud, free mistral model, one gateway file `src\llm\client.py` ; offline rules score first.
- "Where is data stored?" → MongoDB Atlas cloud; if it is unreachable, local SQLite file takes over (`src\storage\database.py`).

- "What is novel here?" → Closed loop: threat → detect → live alert → guided fix (text+voice+video) → saved event → risk visible. One JSON, one DB, one server.
- "How do you know it works?" → `pytest tests -q` → 3 passed; plus live `/health` , Swagger `/docs` , curl lines in `../instructor/API_TABLE.txt` .
- "Limits?" → Prototype realtime = REST+WebSocket (no raw packet sniffing); free Render sleeps ~30s cold start; LLM needs key + net; video = slide guide, not a presenter avatar.

8. Tonight checklist (before sleep)

[] Double-click bat → both links 200 → one Simulate → one support ask [] Read Section 5 recipes once; try E1 + E3 yourself (then undo) [] Charge laptop; hotspot ready (LLM + Atlas need internet) [] Keep this black terminal OPEN or remember: bat → 5 sec → refresh [] `.env` stays on THIS pc; never open it on the projector

Good luck tomorrow. You built a full closed-loop system — show it.